

```

    }

void print()
{
cout<< "In Derived::print() : b_ = " << b_ <<endl;
}

private:
int b_;
};

void disp (Base ob)
{
ob.print();
}

int main()
{
    Base b(10);
    Derived d(15, 25);

disp(b);
disp(d); // slicing will happen

return 0;
}

----- output -----

In Base::print() : a_ 10
In Base::print() : a_ 15    <-- Due to slicing

```

The example below demonstrates object slicing when a derived class object is assigned to the base class object:

```

#include <iostream>
using namespace std;

class Base
{
public:
Base(int a)
{
    a_ = a;
}

virtual void print()
{
cout<< "In Base: a_ = " << a_ <<endl;
}

private:
int a_;
};

```

```

class Derived : public Base
{
public:
Derived(int a, int b):Base(a)
{
    b_ = b;
}

void print()
{
cout<< "In Derived: b_ = " << b_ <<endl;
}

private:
int b_;
};

int main()
{
    Base b(10);
    Derived d(15, 25);

b.print();
d.print();

    b = d;    // Assign Derived class object to Base class
              // object.
b.print();
d.print();

return 0;
}

----- output -----

In Base: a_ = 10
In Derived: b_ = 25
In Base: a_ = 15    <-- This is due to slicing!
In Derived: b_ = 25

```

Of course, we did not define the assignment operator for the base class (which, by the way, is not needed in this case, as the member 'a_' is an 'int' and the compiler performed assignment is good enough), but even that wouldn't have helped here. This is because the object 'd' gets sliced when the assignment operator is invoked! Remember, operator overloading is actually a function call. Therefore, the object 'd', if you realise, will be passed as a function argument. The assignment operator (function) takes this passed object of the derived class as a base class object and so the derived portion is lost while copying.

One could argue that this behaviour seems fair and reasonable by all means. A base class object is equipped only